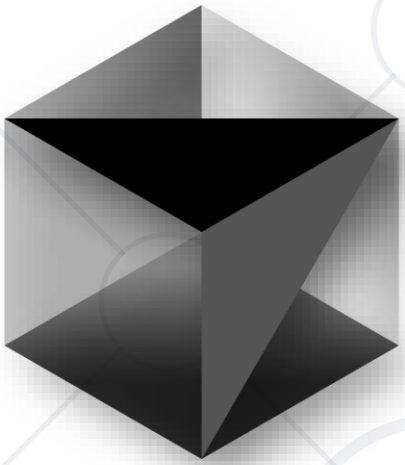


Cursor AI Code Editor



SoftUni Team
Technical Trainers



SoftUni



Software University

<https://softuni.org>

Table of Contents

1. About Cursor
2. Cursor Features
3. Prompting Strategies
4. Working on Legacy vs New Project
5. Modularity with AI





About Cursor IDE

What is Cursor?

- **AI-powered** IDE built on top of VS Code
- Designed for **full-project understanding**
- Strong focus on **code generation, refactoring, and multi-file** edits
- Lightweight, fast, and developer-first
- **Integrates** deeply with Git workflows

- Can choose between all popular models
- **Auto-picks** the best model based on **task type**
- Free tier with limited daily usage
- Pro plan with increased context window and agent cycles
- Option to bring your own API keys for any model

Individual Plans

Hobby
Free

Pro
\$20/mo.

Pro+ **Recommended**
\$60/mo.

Ultra
\$200/mo.

Installation and Setup

- Download Cursor installer for your OS
- Imports most VS Code settings automatically
- Compatible with VS Code extensions
- No complex configuration required
- Sign In



Download Cursor

Available for macOS, Windows, and Linux.

Download for Windows [↓](#)



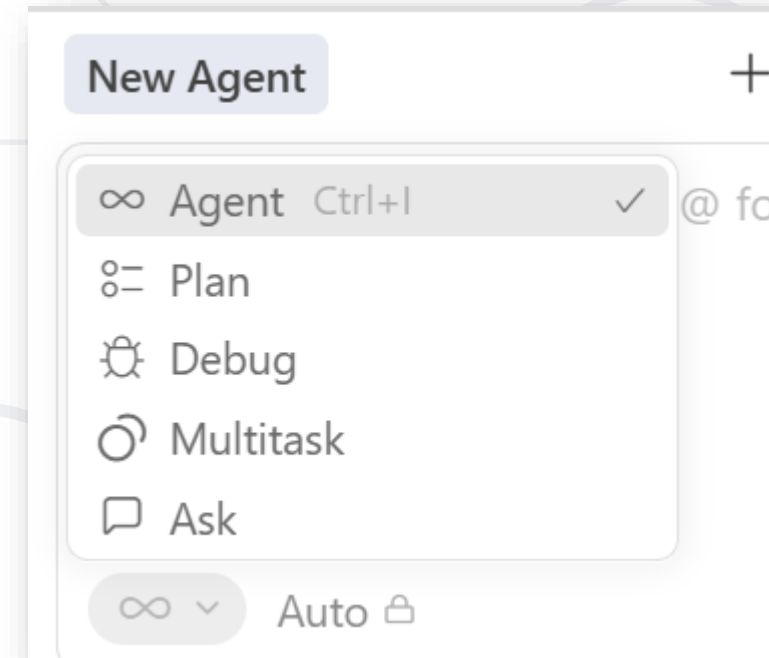
Cursor Features

- Real-time code **completions**
- Generate functions **from comments**
- Multi-line edits **directly** in the **editor**
- **Refactor** or rewrite code in place
- Add **tests** with a single command

Ctrl+L to chat, Ctrl+K to generate

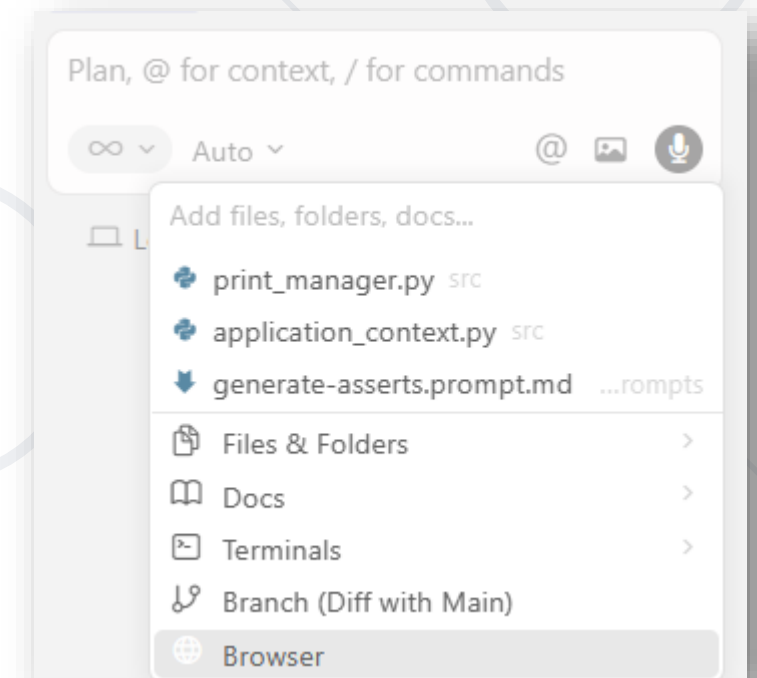
Chat: Agent, Plan, Debug, Ask

- **Ask**: direct one-off instructions
- **Plan**: Cursor proposes multi-step change flow
- **Debug**: Interactive process with multiple hypotheses
- **Agent**: executes the plan automatically
- Ability to **approve**, **tweak**, or **reject** each **step**



- Use **multiple subagents** on the **same task**
- **Break down** larger tasks into **smaller chunks**
- Messages in the **queue can be converted** to multitask

- Reads **entire files** and **folders** automatically
- **Pulls** relevant classes, config, API routes, envs
- **Maintains state** across complex tasks
- Greatly **reduces “hallucinations”** during edits

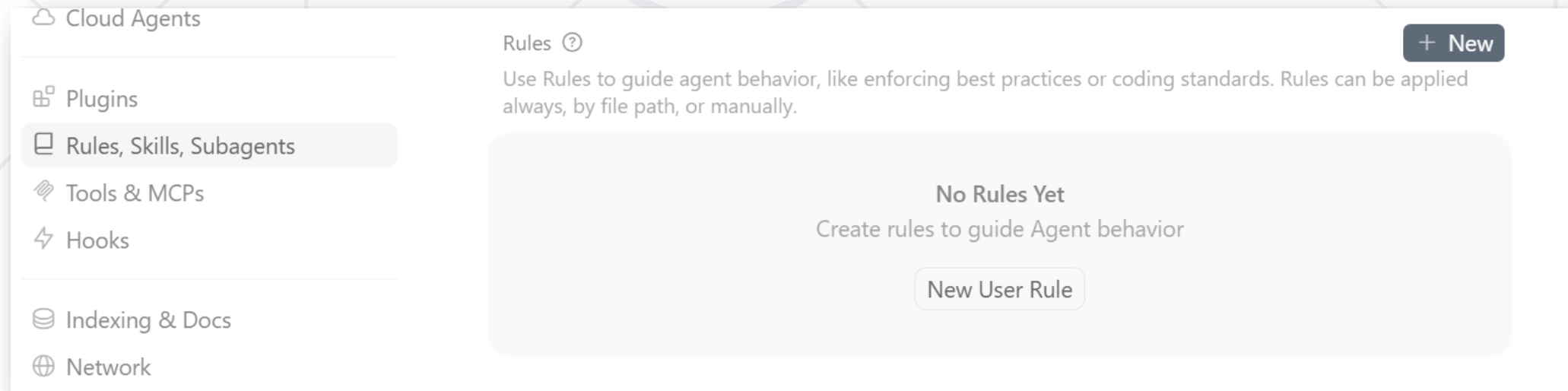


- Upload screenshots or **UI flows**
- Extracts **text**, **structure**, and design intent
- Useful for converting **UI → code** or reverse-engineering components

- Stores **earlier conversations** per project
- AI **remembers** previous architecture/context
- Useful for long-running development sessions

Rules and Commands

- **Rules** – define behavior, coding style, and constraints
- **Commands** – run AI actions directly in the editor



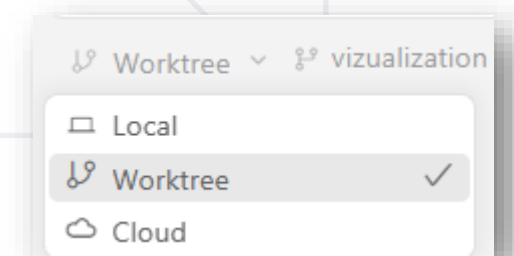


What makes Cursor AI-First?

- **Automatically execute multi-step plans**
- Understand **file dependencies** and **project structure**
- Perform refactors across multiple files
- **Iterate** until the task is **complete** (controlled by cycle count)
- **Developer** can **approve** or modify each step
- Enables “AI as junior developer” workflows

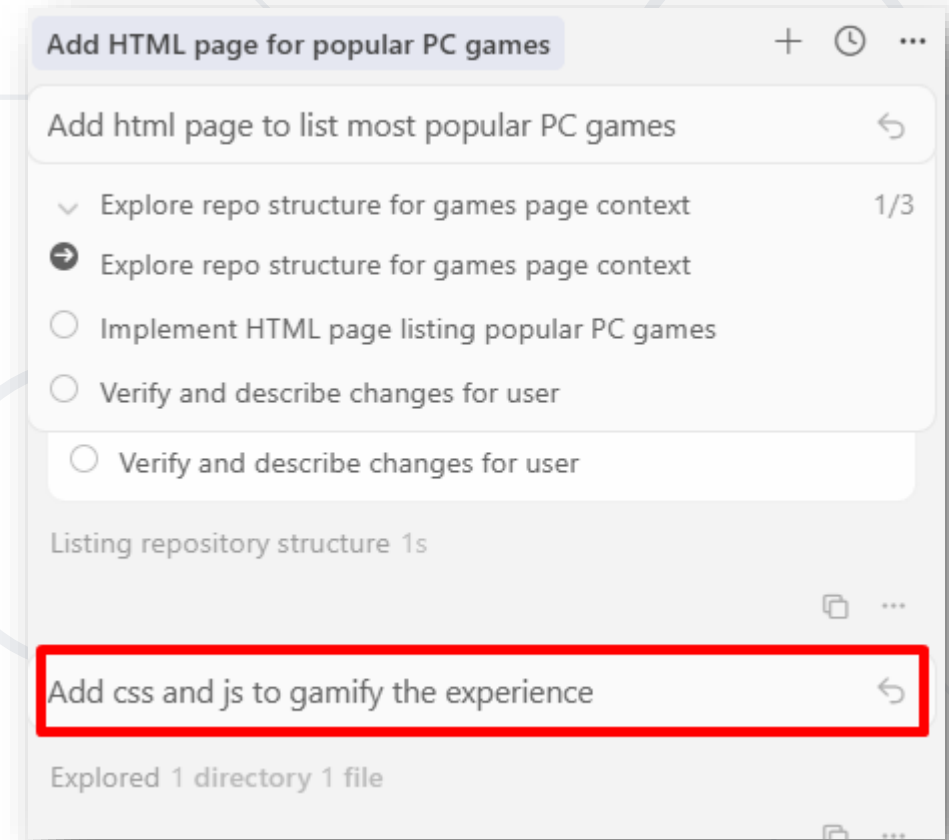
- **AI-first workspace** organized around chat, diffs, and context
- Code, chat, and reasoning are visible **side-by-side**
- **Automatic** file **suggestions** based on current task
- Designed to **keep you in flow** without switching windows

- Creates **isolated AI-generated branches** safely
- Lets you **experiment** without touching the main project
- **Compare**, merge, or discard changes with one click
- Ideal for big refactors or **uncertain tasks**



Message Queue

- Manages AI tasks in a **structured sequence**
- Prevents **conflicting** edits and **overlapping** diffs
- Ensures predictable, **reproducible changes**
- Great for **long, multi-step** modifications



- **Cursor's own** high-level AI **model**
- Designed to reason across the entire project structure
- Builds modules, flows, and scaffolding step-by-step
- Ideal for complex implementations

Mar 19, 2026

Introducing Composer 2

3 min read

Composer 2 is now available in Cursor.

It's frontier-level at coding and priced at \$0.50/M input and \$2.50/M output tokens, making it a new, optimal combination of intelligence and cost. We also released a [technical report](#) on how we trained it.

Comparison with GitHub Copilot

Aspect	Cursor	GitHub Copilot
Core Idea	AI-first IDE built around AI workflows	AI assistant integrated into a familiar IDE
Workflow Style	AI drives big edits; more automation	You stay in control; AI assists step by step
Editing Power	Strong multi-file and project-wide edits	Precise edits on selected code, smaller scope
Autocomplete	More creative and aggressive suggestions	More stable, conservative, language-tight
Refactoring	Better for large, structural refactors	Great for small/medium, local refactors
Chat Context	Deep project understanding, broader scope	Focus on open files + configured context
Ecosystem	Standalone tool, fewer integrations	Rich: VS Code extensions, @-extensions, MCP
Best For	Big changes, architecture, explorations	Daily coding, tests, reviews, PR workflows



Prompting Strategies

- Be **explicit** about goals and constraints
- Reference **files/functions** directly
- Break tasks into **smaller steps**
- Use **follow-up prompts** for refinement

In *auth.py* and *middleware/security.py*, add support for refresh tokens.

- Keep the existing login flow working.
- Update only what's necessary.
- First propose a step-by-step plan, then wait for my approval before changing any code.

- Target specific files for **safe edits**
- Ideal for **small refactors** or bug fixes
- Prevents **accidental** multi-file changes

In *utils/date_formatter.js*, extract the date parsing logic into a new function *parseDate(input)*. Keep behavior identical.

- Use for **new features** or **architecture** changes
- Ask for a **plan** before execution
- Let the **agent handle** multi-file updates

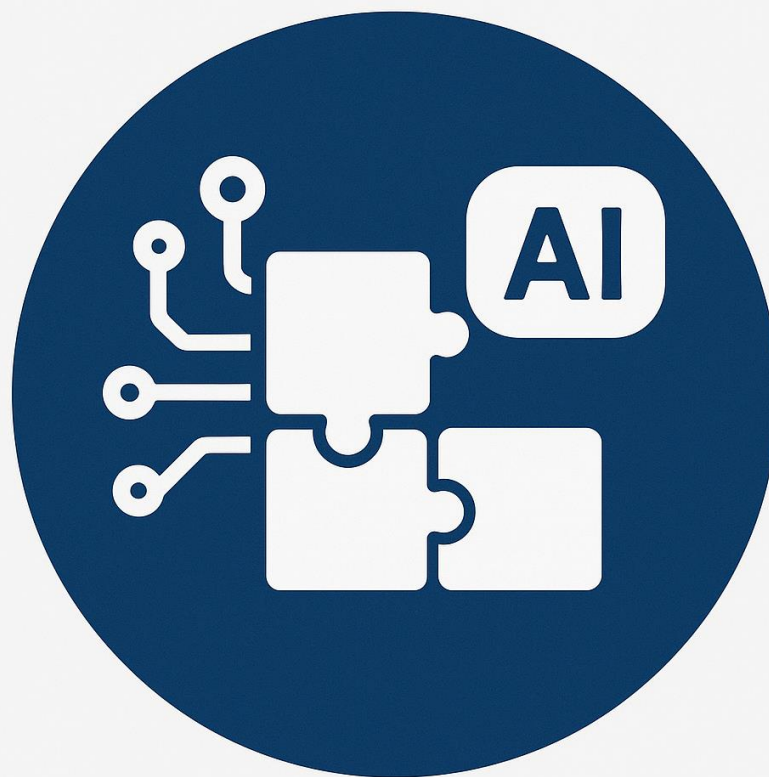
Add JWT authentication across the project. Show me a step-by-step plan before making changes.

- Ask for error **cause**, not just fixes
- Prefer **minimal patches** instead of rewrites
- **Request explanation** first if needed

Here is the stack trace. Explain what caused the error and propose the smallest code change to fix it.

- **Limited context** – AI can't see the entire codebase; you must provide targeted context (Search → Ask → Modify).
- **Old tech stack / versions** – AI must be told NOT to suggest modern patterns that aren't compatible.
- **Technical debt everywhere** – implicit dependencies, global state, side effects, anti-patterns confuse the model.
- **Refactoring risk** – large-scale changes from Plan can easily break things; work in small, safe steps.
- **Local, incremental improvements** – validation, tests, documentation, small refactors, performance fixes.

In a new project, AI acts as the “*architect and builder*”; in a legacy project, it is a “*diagnostician and careful repair technician*.”

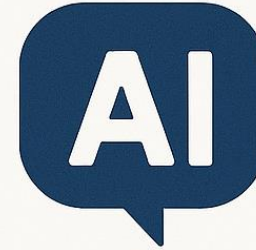


Modularity with AI

Why Modularity Matters

- AI works best with **small, isolated units** of logic
- **Clean boundaries** improve context accuracy
- **Easier** to test, review, and revert generated changes

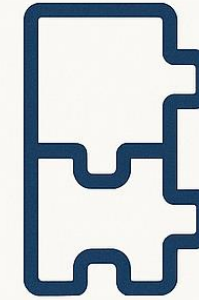
- Ask Cursor to **propose** a **module breakdown**
- Implement each module **iteratively** with **Agents**
- **Validate diffs** after each step
- Maintain a **stable, predictable** project structure



Ask Cursor to propose a module breakdown



Implement each module iteratively with Agents



Validate diffs after each step



Maintain a stable, predictable project structure



Benefits of Modular Development

- **Faster** and **safer** AI-assisted development
- **Reduced risk** of unwanted side effects
- **Easier** debugging and refactoring
- **Higher-quality** generated code overall

- Cursor is an **AI-powered IDE** that understands your entire project and helps you build faster.
- It uses **inline edits**, **chat**, and **multi-step agents** to generate, refactor, and debug code.
- **Clear, structured prompting** leads to more accurate and predictable results.
- **New projects** offer **better context** and more reliable agent behavior than legacy ones.
- **Modularity** makes **AI** collaboration easier, safer, and more scalable.



- Software University – High-Quality Education, Profession and Job for Software Developers

- softuni.bg

- Software University Foundation

- softuni.foundation

- Software University @ Facebook

- facebook.com/SoftwareUniversity



- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – <https://about.softuni.bg>
- © Software University – <https://softuni.bg>

